Semantic Types and Their Vector Counterparts

What Each Montague Type Becomes in a Vector Space

James Pustejovsky

Brandeis University
Department of Computer Science

CS 135: Computational Semantics

Fall 2026

Part I, Pushed As Far As It Goes

Six sessions built a semantics that assigns truth conditions, composes by function application, licenses inference with proof, and predicts syntactic distribution from semantic properties.

It has also been engineered, outside this room, about as hard as anyone has engineered it.

Compact notations	λ-DCS and its relatives strip the variables out and run against a billion facts
Learned from data	semantic parsers trained from question–answer pairs alone, with no annotated logical forms
At scale	Freebase-scale question answering, in production

And *still*, in every one of those systems, the representation of a word is an arbitrary symbol. Scientist and Physicist are two unrelated constants whose denotations happen to overlap.

The gap is not an implementation detail. It is structural.

Which Is Why We Are Not Simply Switching Topics

The obvious next move is to put Part I aside and start over with vectors. We are going to do something more careful first.

Before asking what vectors can do, ask what the logic *already is*, in vector terms.

If we skip this	"logic and vectors are different paradigms" stays a slogan
If we do it	we find out exactly which parts translate, which translate at a cost, and which do not translate at all

Today's answer will be sharper than either camp usually admits: a good deal of Montague semantics is already linear algebra, provably and exactly — and the place it stops is not where you would guess.

Next session, when we build distributional vectors from a corpus, you will be able to say precisely what changed. That is worth one lecture's delay.

The Question

Part I gave every expression a **semantic type**: e, t, $\langle e, t\rangle$, $\langle\langle e, t\rangle, t\rangle$, and so on.

Part II will give every expression a **vector**.

For each type τ, what is the vector-space object?

This is not a rhetorical question. It has a precise answer for some types, a contested answer for others, and no answer at all for a few.

Today we work out which is which – carefully, and with the arithmetic done rather than gestured at.

Two Regimes – Do Not Confuse Them

	Exact	**Approximate**
Entities	one-hot in $\mathbb{R}^n$	dense in $\mathbb{R}^d$, $d \ll n$
Domain	finite, enumerated	arbitrary
The mapping is	a **theorem**	a **design decision**
Truth values	exactly $\{0, 1\}$	degrees in $[0, 1]$
Similarity	none	yes
Verifiable	yes – check it	only empirically

Most confusion in this area comes from sliding between the two columns without saying so. We do the left column first and completely, because it is the only way to see precisely what the right column gives up.

The left column is not a toy. It is exactly what a finite model is, written in linear algebra.

Which Vector Space?

"Vectors" is not one choice. There is a ladder, and each rung adds structure.

Space	Adds	Composition	In this course
$\mathbb{R}^n$	inner product	$\mathbf{p} \cdot \mathbf{q}$, $\mathbf{p} \odot \mathbf{q}$	**today**
$T(\mathbb{R}^n)$	tensor products	index contraction	**today**; DisCoCat
$\Lambda(\mathbb{R}^n)$	antisymmetry	wedge $\wedge$	Sessions 12–13
$\mathrm{Cl}(\mathbb{R}^n)$	one invertible product	geometric product, $\lrcorner$	Sessions 14–15

Today we work on the first two rungs – ordinary linear algebra, extended to tensors for relations of arity > 1. No grades, no wedge, no multivectors.

Why start here: on these rungs the correspondence is *checkable*. Every claim today is arithmetic you can verify by hand, which is the right foundation before we move to a substrate where the claims become proposals.

Two Rungs We Are Skipping, and Why They Exist

Substrate	Types map to	The catch
Hyperdimensional / VSA	$\mathbb{R}^d$ – *all* of them	arity is not recoverable
Clifford $\mathrm{Cl}(\mathbb{R}^n)$	grades	$\mathbf{a} \wedge \mathbf{a} = 0$: no argument reuse

The VSA row is an instructive failure. Fixed dimension is exactly what solves the n^k growth we will hit later today. But it solves it by mapping *every* type to the same space — so the type-to-object correspondence becomes constant, and you cannot read a predicate's arity off its representation.

That is the precise content of the claim, in Session 11, that these architectures are "untyped".

The Clifford row buys coordinate-freedom for both composition rules, and makes grade intrinsic. Its price is antisymmetry: a predicate cannot take the same argument twice without vanishing. That is the reflexive problem, appearing as an algebraic fact rather than a linguistic observation.

The Basis Dependence We Are Accepting

An honest caveat about today's machinery, which we will have to repay.

Operation	**Role**	**Basis-independent?**
Inner product $\mathbf{p} \cdot \mathbf{e}_a$	application	yes (given a metric)
Hadamard $\mathbf{p} \odot \mathbf{q}$	modification	**no**

The Hadamard product is defined *componentwise*, so it depends on which basis you wrote the vectors in. Change basis and $\mathbf{p} \odot \mathbf{q}$ changes meaning.

So one of our two composition rules is not coordinate-free. It works today only because we fixed an enumeration of $\mathcal{D}_e$ and never move off it.

Session 12 opens by insisting that a vector is *not* a list of numbers. When it does, this slide is the debt being called in — and the wedge and contraction of geometric algebra are the repair, since both are defined without reference to coordinates.

Today

1. Base types: e and t
2. First-order types: predicates as indicator vectors, relations as matrices
3. Application is contraction
4. **Generalized quantifiers** – and why conservativity saves us
5. The hierarchy as tensor order
6. Where the exact correspondence breaks
7. The approximate regime, and what it costs
8. Three candidate substrates
9. Scorecard: settled, contested, open

Setup

Fix a finite domain and enumerate it:

$$\mathcal{D}_e = \{d_1, d_2, \ldots, d_n\}$$

Write $\mathcal{V}(\cdot)$ for the map from semantic objects to vector-space objects.

The base cases

$$\mathcal{V}(e) = \mathbb{R}^n, \quad \text{with } \mathcal{V}(d_i) = \mathbf{e}_i \ \ (\text{the } i\text{th standard basis vector})$$

$$\mathcal{V}(t) = \mathbb{R}, \quad \text{with truth} = 1 \text{ and falsity} = 0$$

So an entity is a **one-hot vector**: all zeros except a single 1 in its own position.

Everything today rests on this choice. We revisit it in the second half, where we replace it – and it will turn out that nearly everything that breaks, breaks because of this slide.

An Immediate Consequence

For distinct entities $d_i \neq d_j$:

$$\mathbf{e}_i \cdot \mathbf{e}_j = 0$$

Distinct entities are orthogonal.

Every pair of distinct individuals is exactly as dissimilar as every other pair. *Alice* and *Bob* are as far apart as *Alice* and *the number seven*.

This is the formal statement of what we have been saying since Session 1: the logical tradition has no notion of similarity. It is not an oversight or an omission – it is a geometric fact about the encoding. One-hot vectors carry identity and nothing else.

Note that we have gained something by saying it this way: the deficiency is now measurable. We can ask exactly how much room there is between orthogonality and useful similarity, and Part II's answer is: d dimensions' worth.

Type $\langle e, t\rangle$: Predicates as Indicator Vectors

A predicate of type $\langle e, t\rangle$ denotes a subset of $\mathcal{D}_e$, equivalently a characteristic function.

$$\mathcal{V}(\langle e, t\rangle) = \mathbb{R}^n, \qquad \mathcal{V}(P) = \mathbf{p} \;\text{ with }\; p_i = \begin{cases} 1 & d_i \in P \\ 0 & \text{otherwise} \end{cases}$$

Example. With $\mathcal{D}_e = \{\text{Alice}, \text{Bob}, \text{Carol}, \text{Fido}\}$ and $\text{student} = \{\text{Alice}, \text{Bob}, \text{Carol}\}$:

$$\mathcal{V}(\text{student}) = (1, 1, 1, 0)^\top \qquad \mathcal{V}(\text{sleep}) = (0, 0, 1, 1)^\top$$

Note that $\mathcal{V}(e)$ and $\mathcal{V}(\langle e, t\rangle)$ are *the same space*, $\mathbb{R}^n$. Predicates are the corners of the cube $\{0, 1\}^n$; entities are the corners with exactly one 1. The type distinction survives as a distinction between *which* vectors, not *which space*.

Application Is an Inner Product

$$[\![P]\!]([\![a]\!]) \;=\; 1 \quad \text{iff} \quad a \in P \quad \text{iff} \quad \mathbf{p} \cdot \mathbf{e}_a = 1$$

Semantics	**Vector operation**	**Result type**
$[\![P]\!]([\![a]\!])$	$\mathbf{p} \cdot \mathbf{e}_a$	scalar $\in \mathbb{R} = \mathcal{V}(t)$

Check it. $\mathcal{V}(\text{sleep}) = (0,0,1,1)$ and $\mathcal{V}(\text{Alice}) = (1,0,0,0)$:

$$(0,0,1,1) \cdot (1,0,0,0) = 0 \qquad \textit{Alice sleeps} \text{ is false } \checkmark$$

$$(0,0,1,1) \cdot (0,0,1,0) = 1 \qquad \textit{Carol sleeps} \text{ is true } \checkmark$$

The inner product against a one-hot vector is *lookup*. Function application has become indexing.

Boolean Operations Are Vector Operations

Semantics	**Set**	**Vector**
Predicate modification	$P \cap Q$	$\mathbf{p} \odot \mathbf{q}$ (Hadamard)
Disjunction	$P \cup Q$	$\mathbf{p} + \mathbf{q} - \mathbf{p} \odot \mathbf{q}$
Negation	$\mathcal{D}_e \setminus P$	$\mathbf{1} - \mathbf{p}$
Subset	$P \subseteq Q$	$\mathbf{p} \odot \mathbf{q} = \mathbf{p}$
Cardinality	$\lvert P \rvert$	$\mathbf{p} \cdot \mathbf{1}$

Example. *sleeping student*:

$$(1,1,1,0) \odot (0,0,1,1) = (0,0,1,0) = \mathcal{V}(\{\text{Carol}\}) \quad \checkmark$$

Predicate modification – our second composition rule from Session 2 – is componentwise multiplication. That is a genuinely tidy result, and it holds exactly.

The last row will matter later: counting is an inner product with the all-ones vector.

Type $\langle e, \langle e, t\rangle\rangle$: Relations as Matrices

A transitive verb denotes a set of pairs.

$$\mathcal{V}(\langle e, \langle e, t\rangle\rangle) = \mathbb{R}^{n\times n}, \qquad M_{ij} = \begin{cases} 1 & (d_i, d_j) \in R \\ 0 & \text{otherwise} \end{cases}$$

The same relation you used this morning: like $= \{(\mathsf{A}, \mathsf{B}), (\mathsf{B}, \mathsf{C}), (\mathsf{C}, \mathsf{F}), (\mathsf{F}, \mathsf{F})\}$, ordered A, B, C, F:

$$M_{\text{like}} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix} \qquad \text{row} = \text{liker},\ \text{column} = \text{liked}$$

Two applications, two operations:

$$[\![\text{likes}]\!]([\![\text{Bob}]\!]) = M\mathbf{e}_{\mathsf{B}} \quad \text{– a column: the VP } \textit{likes Bob}\text{, of type } \langle e, t\rangle$$

$$[\![\text{Alice likes Bob}]\!] = \mathbf{e}_{\mathsf{A}}^{\top} M \mathbf{e}_{\mathsf{B}} \quad \text{– a scalar: a bilinear form}$$

Currying Determines Contraction Order

$$\mathbf{e}_{\mathsf{A}}^{\top} M \mathbf{e}_{\mathsf{B}} \qquad \text{in index notation} \qquad a^i \, M_{ij} \, b^j$$

Two indices, both summed. **Two contractions, in a fixed order** – exactly the two β-reductions of Session 1.

Step	Lambda calculus	Linear algebra	
1	apply to object	contract index j:	$M\mathbf{b}$
2	apply to subject	contract index i:	$\mathbf{a}^{\top}(M\mathbf{b})$

Currying is the statement that contractions happen one at a time, in a determined order. The type $\langle e, \langle e, t \rangle \rangle$ records that order; the matrix alone does not.

Which is why M and $M^{\top}$ are different verbs, and why we insisted in Session 2 that the object comes first. In this morning's terms: transposition is re-currying, and $M\mathbf{e}_j$ is the one that matches $[\![\text{like}]\!](j)$.

The Recurring Operation

We have now met contraction under five names.

Tradition	**Operation**	**The eliminated position**
Lambda calculus	β-reduction	the bound variable
Predicate logic	$\exists$-elimination	the quantified variable
λ-DCS †	join $B.U$	implicit in the operation
Linear algebra	$M\mathbf{u}$	the summation index
Geometric algebra	$A \lrcorner B$	the grade that is lowered

One operation: combine two structures along a shared position, and eliminate that position. Function application is contraction — the reason this correspondence works at all, and what Part IV builds on.

† λ-DCS: a variable-free query language for QA over knowledge bases (Liang 2013). Its one composition rule, *join*, combines two sets along a shared argument and existentially closes it — the same move, variable never written.

Practice: Compute by Hand

Using $\mathcal{D}_e = \{\mathsf{A}, \mathsf{B}, \mathsf{C}, \mathsf{F}\}$ in that order, with

$$\mathcal{V}(\mathsf{student}) = (1,1,1,0), \quad \mathcal{V}(\mathsf{dog}) = (0,0,0,1), \quad \mathcal{V}(\mathsf{sleep}) = (0,0,1,1)$$

1. $\mathcal{V}(\textit{sleeping dog})$
2. Is *Carol sleeps* true? Give the inner product.
3. $|\mathsf{student}|$, as a vector operation
4. Is *some dog sleeps* true? Use the determiner test.
5. $\mathcal{V}(\textit{non-sleeping student})$

Practice: Compute by Hand

Using $\mathcal{D}_e = \{\mathsf{A}, \mathsf{B}, \mathsf{C}, \mathsf{F}\}$ in that order, with

$$\mathcal{V}(\mathsf{student}) = (1,1,1,0), \quad \mathcal{V}(\mathsf{dog}) = (0,0,0,1), \quad \mathcal{V}(\mathsf{sleep}) = (0,0,1,1)$$

1. $\mathcal{V}(\textit{sleeping dog})$
2. Is *Carol sleeps* true? Give the inner product.
3. $|\mathsf{student}|$, as a vector operation
4. Is *some dog sleeps* true? Use the determiner test.
5. $\mathcal{V}(\textit{non-sleeping student})$

Answers.

1. $(0,0,0,1) \odot (0,0,1,1) = (0,0,0,1)$ – just Fido
2. $(0,0,1,1) \cdot (0,0,1,0) = 1$ true
3. $\mathbf{a} \cdot \mathbf{1} = 3$
4. $\mathbf{a} \cdot \mathbf{b} = (0,0,0,1) \cdot (0,0,1,1) = 1 \geq 1$ true
5. $(1,1,1,0) \odot \big(\mathbf{1} - (0,0,1,1)\big) = (1,1,1,0) \odot (1,1,0,0) = (1,1,0,0)$

The Same Sentence, Four Ways

Alice likes Bob.

Session 1 – lambda calculus	$(\lambda x.\,\mathsf{like}(x,\mathsf{b}))(\mathsf{a}) \rightsquigarrow \mathsf{like}(\mathsf{a},\mathsf{b})$
Session 3 – model theory	$(\mathsf{A},\mathsf{B}) \in I(\mathsf{like})$
Session 4 – Haskell	`spName alice (vpName likes bob)`
Today – linear algebra	$\mathbf{e}_{\mathsf{A}}^{\top} M_{\mathsf{like}}\, \mathbf{e}_{\mathsf{B}} = 1$

Four notations, one computation. Each makes something different easy: the first makes composition explicit, the second makes truth explicit, the third makes it executable, and the fourth makes it **differentiable**.

That last property is the one Part II needs and the others cannot supply. Nothing else about the fourth row is new – and that is exactly the point.

The Hard Case

$$\textit{every student} \;:\; \langle\langle e, t\rangle, t\rangle$$

A function from predicates to truth values. Its domain is $\mathcal{D}_{\langle e,t\rangle}$, which has 2^n elements.

Type	Domain size	Naive encoding
e	n	$\mathbb{R}^n$
$\langle e, t\rangle$	2^n	$\mathbb{R}^n$ (indicator)
$\langle\langle e, t\rangle, t\rangle$	2^{2^n}	$\mathbb{R}^{2^n}$

For $n = 4$ that is $\mathbb{R}^{16}$ – fine. For $n = 100$ it is $\mathbb{R}^{2^{100}}$.

The naive encoding is hopeless. A generalized quantifier is a set of sets, and there is no room to write one down.

And yet natural language uses these constantly, in bounded working memory. Something must be special about the quantifiers that actually occur.

Three Universals

Not every function of type $\langle\langle e, t\rangle, t\rangle$ is a possible determiner meaning. Three constraints hold across languages.

Conservativity (CONS)

$$D(A)(B) \iff D(A)(A \cap B)$$

Only the part of B inside A matters. *Every student sleeps* says nothing about non-students.

Extension (EXT)

D does not depend on entities outside $A \cup B$. Enlarging the domain with irrelevant objects changes nothing.

Isomorphism-invariance (ISOM)

D depends only on *cardinalities*, not on which particular individuals are involved. Permuting the domain does not change the truth value.

What the Universals Buy

Take the three together.

- CONS: only A and $A \cap B$ matter.
- EXT: nothing outside $A \cup B$ matters.
- ISOM: only sizes matter, not identities.

Therefore D is a function of exactly **two numbers**:

$$D(A)(B) \;=\; f\big(\,|A - B|,\; |A \cap B|\,\big)$$

From 2^{2^n} possibilities down to a function on a pair of integers. That is an enormous compression, and it is forced by empirical facts about natural language rather than chosen for convenience.

This is the standard result of generalized quantifier theory – Barwise & Cooper (1981), Keenan & Stavi (1986), van Benthem (1986).

And Both Numbers Are Inner Products

For indicator vectors $\mathbf{a}, \mathbf{b} \in \{0,1\}^n$:

$$|A \cap B| = \mathbf{a} \cdot \mathbf{b} \qquad |A - B| = \mathbf{a} \cdot \mathbf{1} - \mathbf{a} \cdot \mathbf{b}$$

Every conservative, ISOM determiner is a numeric test on $\mathbf{a} \cdot \mathbf{b}$ and $\mathbf{a} \cdot \mathbf{1}$.

Determiner	Condition
every	$\mathbf{a} \cdot \mathbf{1} - \mathbf{a} \cdot \mathbf{b} = 0$
some, a	$\mathbf{a} \cdot \mathbf{b} \geq 1$
no	$\mathbf{a} \cdot \mathbf{b} = 0$
at least k	$\mathbf{a} \cdot \mathbf{b} \geq k$
exactly k	$\mathbf{a} \cdot \mathbf{b} = k$
most	$\mathbf{a} \cdot \mathbf{b} > \mathbf{a} \cdot \mathbf{1} - \mathbf{a} \cdot \mathbf{b}$

The Payoff, Stated Plainly

	Dimensions needed
Naive encoding of $\langle\langle e, t\rangle, t\rangle$	2^n
With CONS + EXT + ISOM	**2**

The linguistic universals are exactly what makes the vector encoding tractable.

This is worth sitting with. Conservativity is usually presented as a descriptive generalization about determiner meanings – a curiosity of quantifier theory.

Here it turns out to be a **compression theorem**. The reason quantification can be represented in bounded space at all is that natural language only uses the quantifiers that compress.

Whether that is a coincidence, or whether languages have the quantifiers they do *because* those are the representable ones, is a genuinely open question – and a good essay topic.

Worked: *Every student sleeps*

$$\mathbf{a} = \mathcal{V}(\text{student}) = (1, 1, 1, 0) \qquad \mathbf{b} = \mathcal{V}(\text{sleep}) = (0, 0, 1, 1)$$

Compute the two numbers.

$$\mathbf{a} \cdot \mathbf{b} = 1 \ (|A \cap B|, \text{ only Carol}) \qquad \mathbf{a} \cdot \mathbf{1} = 3 \ (|A|) \qquad \mathbf{a} \cdot \mathbf{1} - \mathbf{a} \cdot \mathbf{b} = 2 \ (|A - B|)$$

Apply the tests.

every student sleeps	$2 = 0$?	**false**
some student sleeps	$1 \geq 1$?	**true**
no student sleeps	$1 = 0$?	**false**
most students sleep	$1 > 2$?	**false**

All four agree with the Session 3 evaluations. ✓ And every one is two inner products and a comparison.

The Type Hierarchy as Tensor Order

Type	Object	Order	Example
t	scalar	0	a truth value
e	vector $\in \mathbb{R}^n$	1	*Alice*
$\langle e, t\rangle$	vector $\in \mathbb{R}^n$	1	*sleeps*, *student*
$\langle e, \langle e, t\rangle\rangle$	matrix $\in \mathbb{R}^{n\times n}$	2	*likes*
$\langle e, \langle e, \langle e, t\rangle\rangle\rangle$	3-tensor	3	*gave*
$\langle \langle e, t\rangle, \langle e, t\rangle\rangle$	matrix	2	*quickly*
$\langle \langle e, t\rangle, t\rangle$	2 numbers (with universals)	—	*every student*

The pattern for first-order types: tensor order = number of e-slots the type contains.

This is the first honest motivation for "grade as arity". An n-place predicate is an order-n tensor, and applying it lowers the order by one. In Part IV, "order" becomes "grade" and the contraction becomes $\lrcorner$ – but the arithmetic is the arithmetic on this slide.

The Awkward Row

The generalized quantifier row does not fit the pattern, and that matters.

First-order types	order = arity; a uniform rule
$\langle\langle e, t\rangle, t\rangle$	needs the universals; not a tensor of any order

Two honest ways to describe the situation:

- **Optimistic.** Higher types are representable whenever they satisfy strong enough invariances, and natural language supplies those invariances.
- **Pessimistic.** The uniform correspondence covers only first-order types. Above that, every case needs its own argument.

Both are true. The correspondence is clean and mechanical up to relations, and thereafter it works because of facts about language rather than facts about vector spaces.

Five Limits of the Exact Correspondence

1. **Dimension.** First-order types cost n^k for arity k. Higher types cost 2^n absent universals strong enough to compress them.
2. **No similarity.** One-hot entities are orthogonal. This is the whole reason Part II exists.
3. **Finiteness.** The construction enumerates $\mathcal{D}_e$. Infinite domains have no one-hot encoding.
4. **No intensionality.** $\mathcal{V}(P)$ is an extension. Two predicates true of the same things get the *same* vector – so *creature with a heart* and *creature with a kidney* are identical here.
5. **Predicate = extension.** The deep one. The encoding works because a predicate *is* the list of its truth values. That identification is what fails first when we leave the exact regime.

Note that (2) and (5) are the same fact from two directions. Similarity requires that a predicate be something other than its extension – because two predicates can be extensionally identical here and intuitively unalike.

Changing the Base Case

Everything so far followed from $\mathcal{V}(d_i) = \mathbf{e}_i$. Now change it.

$$\mathcal{V}(e) = \mathbb{R}^d, \qquad d \ll n, \qquad \mathcal{V}(d_i) = \mathbf{v}_i \text{ dense, learned}$$

Immediate consequences:

- Entities are **no longer orthogonal**: $\mathbf{v}_i \cdot \mathbf{v}_j \neq 0$ in general.
- That inner product is now *similarity* – the thing the exact regime could not express.
- There is no longer a coordinate per individual, so a predicate **cannot** be the list of its values.

Every difference between the two regimes traces back to this one substitution. It is worth being able to derive the consequences rather than memorizing them.

What the Types Become

Type	Exact	Approximate
e	one-hot $\in \mathbb{R}^n$	dense $\in \mathbb{R}^d$
t	$\{0, 1\}$	$[0, 1]$ – a degree
$\langle e, t\rangle$	indicator $\in \{0, 1\}^n$	functional $\mathbf{w} \in \mathbb{R}^d$, or an MLP
$\langle e, \langle e, t\rangle\rangle$	$\{0, 1\}^{n\times n}$	$W \in \mathbb{R}^{d\times d}$, low rank
$\langle\langle e, t\rangle, t\rangle$	2 numbers	no standard answer

Application survives, with a new reading: $\sigma(\mathbf{w} \cdot \mathbf{v}_a) \in [0, 1]$

Exact	$\mathbf{p} \cdot \mathbf{e}_a$ asks: *is a in P?*
Approximate	$\mathbf{w} \cdot \mathbf{v}_a$ asks: *how much does a look like a P?*

A predicate has become a *hyperplane* rather than a subset: a direction in space, with a score for every point.

The Trade, Made Explicit

	Exact	Approximate
Similarity between entities	—	✓
Graded truth	—	✓
Learnable from data	—	✓
Differentiable	—	✓
Compact ($d \ll n$)	—	✓
Exact application	✓	—
Exact Boolean operations	✓	—
Guaranteed truth conditions	✓	—
Quantifiers	✓	—
Verifiable	✓	—

A clean split – and the bottom block is not a list of engineering shortcomings. It is what you *necessarily* lose when a predicate stops being its extension.

The quantifier row is sharpest: conservativity needed $\mathbf{a} \cdot \mathbf{b}$ to be a *count*. With dense vectors it is a similarity score, and $|A \cap B|$ has no counterpart.

Three Candidate Substrates

Everyone in Part III and Part IV is answering the same question: *can we get the top block without giving up the bottom one?*

Approach	**Types become**	**Composition**
DisCoCat	tensor orders	tensor contraction
Functional GA	grades	left contraction $\lrcorner$
Neural	nothing explicit	learned

One criterion separates them: **does the type structure survive?**

- DisCoCat: yes, explicitly – a transitive verb *is* an order-3 tensor. Cost: d^3 parameters per verb.
- FGA: yes, as grade – an n-place predicate is a grade-n multivector. Cost: an open question about whether grade carries enough.
- Neural: no. Composition is a learned function with no type discipline. Cost: no guarantee that it composes at all.

The Question Part IV Asks

Today's table	type $\to$ tensor order, exactly, for first-order types
Part IV's proposal	type $\to$ grade, over a graded algebra

Grade behaves like order: the wedge raises it, contraction lowers it, and application lowers it by exactly one.

But grade is a single number. It records *how many* arguments, not *what kind*. The types $\langle e, t\rangle$ and $\langle t, t\rangle$ have the same arity and different sorts, and Montague's system keeps them apart.

Whether sortal information can be recovered geometrically – from subspace structure, basis sorting, or signature – is **open**, and a candidate final project.

We flagged this in Session 2. You now have the machinery to see precisely what would have to be shown.

Scorecard

Type	Status	Correspondence
t	settled	scalar
e	settled	vector
$\langle e, t\rangle$	settled	vector; application is inner product
$\langle e, \langle e, t\rangle\rangle$	settled	matrix; application is contraction
n-ary predicates	settled	order-n tensor
$\langle\langle e, t\rangle, t\rangle$	contested	2 numbers *with* universals; open without
Intensional types	open	extensions collapse; no standard treatment
Binding, reuse	open	reflexives, anaphora
Sortal typing	open	grade gives arity, not sort

The top block is not a proposal – it is arithmetic, and you can check every row. The bottom block is where the research is.

You have already met the "binding, reuse" row: $\lambda x.\mathrm{like}(x, x)$ is $\mathrm{diag}(M)$, and reading two indices as one is not a contraction. That is why the row is open rather than merely unfinished.

The Question You Should Leave With

Every number on every slide today came from one place.

$$\mathcal{M} = \langle \mathcal{D}_e,\ I \rangle$$

A finite domain someone enumerated, and an interpretation function someone stipulated. The indicator vectors were read off I. The matrices were read off I. The quantifier's two numbers were computed from sets that I defined.

Now take the model away.

You have a corpus. Nobody has told you what the entities are, what the predicates denote, or which sentences are true.

Where do the numbers come from?

That is the whole of next session, and the answer is not a smaller version of today's. It is a different kind of answer, bought at a price we will state exactly.

Summary

Point	Detail
Two regimes	exact (one-hot, provable) vs. approximate (dense, designed)
e, t	basis vector; scalar
$\langle e, t \rangle$	indicator vector; application is $\mathbf{p} \cdot \mathbf{e}_a$
Modification	Hadamard product $\mathbf{p} \odot \mathbf{q}$
Relations	matrices; $\mathbf{a}^\top M \mathbf{b}$
Currying	fixes the *order* of contraction
Quantifiers	2^n naively; 2 numbers under CONS + EXT + ISOM
The universals	are a **compression theorem**, not a curiosity
Order = arity	for first-order types; motivates grade-as-arity
Orthogonality	one-hot entities are equidistant: no similarity, by construction
The trade	similarity and learnability, against exactness and quantifiers

Reading and Exercises

Reading:

- Barwise & Cooper (1981), *Generalized Quantifiers and Natural Language* – the source of conservativity
- Peters & Westerståhl (2006), *Quantifiers in Language and Logic*, Chs. 3–4
- Widdows (2004), *Geometry and Meaning*, Chs. 1–3

Exercises:

1. Verify the four determiner tests against the Problem Set 1 model, by hand.
2. Give the vector form of *exactly two* and of *between three and five*.
3. Show that *only* is **not** conservative — *only students sleep* $\neq$ *only students are students who sleep*. What does this do to the two-number encoding?
4. Harder. Compute the exact matrix M_{like} for the PS 1 model and check $\mathbf{a}^\top M \mathbf{b}$ against your Haskell results for three sentences.

Exercise 3 is the interesting one: *only* is standardly analyzed as not a determiner at all, and the vector encoding gives an independent reason why.

Questions?

Next: where the numbers come from —
the distributional hypothesis, co-occurrence, and PPMI